

A PROJECT REPORT ON
Stock Market Prediction & Analysis Platform

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD
OF DEGREE OF

B.TECH IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

Caviraj

Register No: **922524243023**



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
V.S.B ENGINEERING COLLEGE, KARUR

CANDIDATE DECLARATION

I, **Caviraj** (Register No: **922524243023**), student of B.Tech in Artificial Intelligence and Data Science at **V.S.B ENGINEERING COLLEGE, KARUR**, hereby declare that the project work entitled "**Stock Market Prediction & Analysis Platform**" is an authentic record of my own work carried out under the guidance of the faculty of the department. This project has not been submitted elsewhere for the award of any other degree, diploma, or title.

Place: Karur

Date:

Signature of Student

DEPARTMENT CERTIFICATE

This is to certify that the project report entitled "**Stock Market Prediction & Analysis Platform**" is a bona fide record of the work done by **Caviraj** (Register No: **922524243023**) in partial fulfillment of the requirements for the award of the degree of B.Tech in Artificial Intelligence and Data Science during the academic year 2025 - 2026.

Head of Department

Internal Examiner

Page 2

Artificial Intelligence and Data
Science

ACKNOWLEDGMENT

I express my sincere gratitude to the Management and Principal of **V.S.B ENGINEERING COLLEGE,KARUR** for providing the necessary facilities and resources to successfully execute this project.

I extend my heartfelt thanks to the Head of Department, **Artificial Intelligence and Data Science**, and all the teaching and non-teaching faculty members of the department who directly or indirectly extended their valuable suggestions and support during the design and development of this platform.

TABLE OF CONTENTS

SECTION / TITLE	PAGE NO
1. Candidate Declaration & Institution Certificate	2
2. Acknowledgment & Table of Contents	3
3. Section 1: Introduction (Background, Scope, Problem Statement)	4
4. Section 2: Project Objectives & System Design Requirements	5
5. Section 3: System Modules & Component Description	6
6. Section 4: System Architecture Diagram & Data Flow	7
7. Section 5: Database Schema Design & Table Relationships	8
8. Section 6.1: Implementation - Deep Learning LSTM Model Training	9
9. Section 6.2: Implementation - Technical Indicators Engine	10
10. Section 6.3: Implementation - FastAPI REST Router Service	11
11. Section 7.1: Output Results - Real-Time Dashboard & Watchlist UI	12
12. Section 7.2: Output Results - LSTM Stock Forecasting & Sentiment Module	13

SECTION / TITLE	PAGE NO
V.S.B Engineering College	Page 3
13. Section 8: Conclusion, Project Scope & References	14

1. INTRODUCTION

1.1 Background & Context

The financial markets are complex, non-linear, and highly dynamic systems influenced by numerous micro and macro-economic factors. Over the past decade, the integration of computational algorithms, statistical modelling, and machine learning has transformed financial analysis. Traditional systems relied purely on human intuition and basic charting tools, but modern investment practices demand quantitative, data-driven strategies capable of processing millions of data points in real time.

This project, titled "**Stock Market Prediction & Analysis Platform**", addresses this need by providing an integrated, full-stack environment where users can track historical financial asset prices, view automated technical indicator calculations, analyze public market sentiment extracted from news streams, and view predictive model outcomes trained via machine learning algorithms.

1.2 Problem Statement

Retail investors face significant barriers when attempting to perform comprehensive financial analysis:

- **Fragmented Tools:** Charting, technical analysis, news sentiment, and statistical forecasting exist as isolated services, requiring users to switch between multiple applications.
- **ML Complexity:** Constructing, training, and running complex time-series neural networks (like Long Short-Term Memory models) requires programming expertise and specialized computing environments.
- **Lack of Real-Time Analysis:** Traditional platforms offer static data instead of recalculating indicators dynamically based on the latest market movements.

1.3 Scope of the Project

The scope of this project involves developing a modular full-stack application. It features a lightweight, high-performance visual dashboard (built with Angular and TradingView Charts), connected to a robust ASGI REST API (built with FastAPI). The platform fetches financial feeds from Yahoo Finance, caches active tickers in an SQLite database, calculates technical metrics, computes

news sentiment scoring, and runs pre-trained LSTM models to forecast asset prices up to 30 days into the future.

2. PROJECT OBJECTIVES & REQUIREMENTS

2.1 Project Objectives

The primary goals of the Stock Market Prediction & Analysis Platform are:

1. **Unified Analytics:** Consolidate charting, indicators, and ML forecasting into a single dashboard.
2. **Deep Learning Integration:** Build and evaluate a recurrent neural network (specifically an LSTM model) to forecast price trends over a 7-day to 30-day forecast horizon.
3. **Algorithmic Indicators:** Automatically compute key technical metrics (SMA, EMA, RSI, Bollinger Bands, MACD) directly on data feeds.
4. **Market Sentiment Analysis:** Extract sentiment from news feeds to gauge public interest and emotional direction for selected tickers.
5. **Resource Efficiency:** Cache downloaded data and model structures locally in an optimized database to prevent redundant API queries.

2.2 System Design Requirements

Functional Requirements:

- Users must be able to search for specific stock tickers (e.g., AAPL, MSFT, GOOGL).
- The platform must render real-time candlestick charts and overlay technical indicator lines dynamically.
- The system must run forecasting models and return predictions along with model parameters.
- Users must have a watchlist page linked to their secure user profile.

Non-Functional Requirements:

- **Performance:** Backend endpoints must respond in less than 500 milliseconds (excluding initial cold-start model loads).
- **Security:** User authentication details must be securely stored using bcrypt hashing, and API communication secured via JWT tokens.

- **Responsiveness:** The user interface must adapt gracefully to different display sizes (desktops, laptops, and tablets).

3. SYSTEM MODULES & COMPONENT DESCRIPTION

The platform architecture is divided into five specialized, decoupled modules. This design isolates data streams, modeling pipelines, and visual layout concerns.

3.1 User Interface Dashboard Module

Developed in **Angular 17.3**, this component serves as the presentation layer. It manages client-side routing, user session state, and watchlist subscriptions. The charting panel embeds the **TradingView Lightweight Charts** library, which draws canvas-based candlestick lines and volume indicators, providing a responsive desktop workspace.

3.2 REST API Server Module

Constructed using the **FastAPI** framework in Python. It handles CORS request policies, parses incoming JSON schemas, verifies JWT credentials, and exposes REST endpoints under ``/stock``, ``/predict``, ``/indicators``, and ``/auth``. Its asynchronous ASGI design guarantees high concurrent connection throughput.

3.3 Machine Learning Pipeline

A dedicated data science pipeline containing three forecasting sub-engines:

- **Deep Learning (LSTM & GRU):** Implemented in Keras/TensorFlow. It scales price trends via MinMaxScaler, passes a 60-day historical window into recursive layers, and outputs forecasted pricing steps.
- **Classical ML (Random Forest, XGBoost):** Computes regression offsets to predict direction trends and generate BUY/HOLD/SELL signals.
- **Statistical Models (ARIMA, SARIMAX):** Provides classical time-series statistical baselines.

3.4 Sentiment Analysis Module

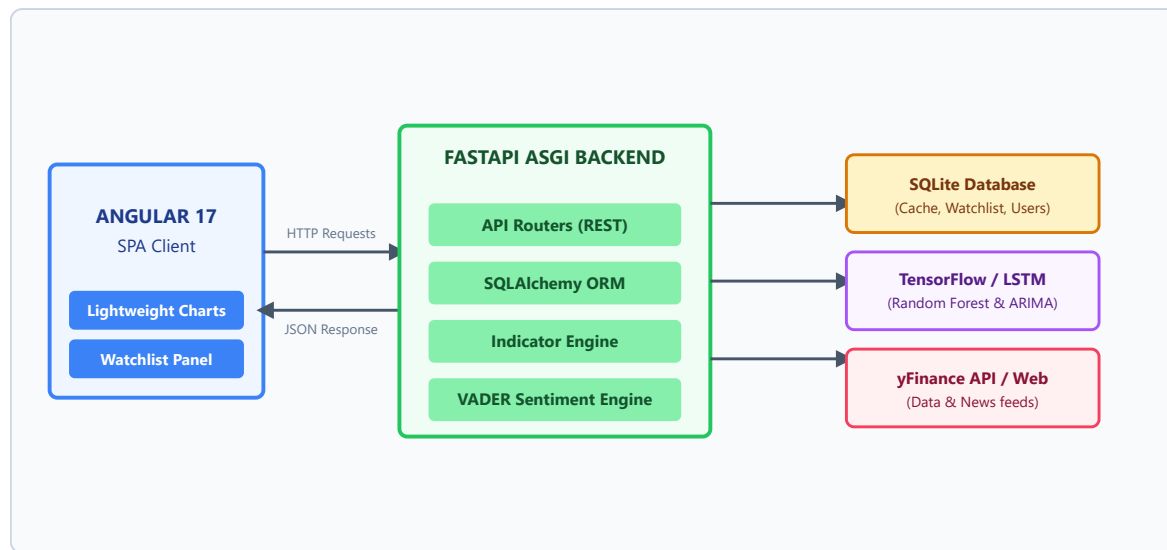
Fetches stock-related news articles via public aggregators, extracts titles and descriptions, and runs them through the **VADER Sentiment Analyzer**. It aggregates compound scores to generate sentiment indicators.

3.5 Database Cache & Authentication Layer

Utilizes **SQLite** with **SQLAlchemy ORM**. It stores encrypted credentials, logs stock prediction histories, and acts as a local Price Cache to prevent Yahoo Finance API rate limits.

4. SYSTEM ARCHITECTURE & DATA FLOW

The diagram below displays the system components and visualizes the network and data boundaries. Communication occurs via HTTP protocols, and local databases cache information for high-speed operation.



4.2 Data Processing & Communication Flow

The data workflow consists of three stages:

- 1. Ingestion & Caching:** A query for a ticker initiates a lookup in the SQLite local `price\_cache` table. If cached records are present and fresh, they are returned. Otherwise, the system fetches historical data from Yahoo Finance, saves it in the database cache, and serves the client.
- 2. Feature Extraction:** Historical prices are converted into Pandas DataFrames, and indicators (such as RSI, SMA, and Bollinger Bands) are calculated.

3. Inference & Pipeline: Scaled datasets are fed into the trained LSTM model to output sequential forecasts, which are serialized to JSON and stored in the `predictions` table.

5. DATABASE DESIGN & SCHEMA

The SQLite database serves as the local data layer. It caches stock prices to decrease latency and stores user watchlist selections and prediction records.

5.1 Users Table (`users`)

Stores authentication credentials and account registration timestamps.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each user.
name	VARCHAR(100)	NOT NULL	Full name of the user.
email	VARCHAR(255)	UNIQUE, INDEX, NOT NULL	Login credential.
hashed_password	VARCHAR(255)	NOT NULL	Bcrypt hashed password.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp.

5.2 Watchlist Table (`watchlist`)

Stores user-configured watchlist connections.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
id	INTEGER	PRIMARY KEY	Unique identifier for the row.
user_id	INTEGER	FOREIGN KEY (users.id)	Link to the user.
ticker	VARCHAR(20)	NOT NULL	Stock ticker code (e.g., AAPL).
added_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp when added.

** Note: A composite unique constraint `(user\_id, ticker)` ensures user watchlists have unique entries.*

5.3 Predictions Table (`predictions`)

Caches predictions generated by machine learning models.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
id	INTEGER	PRIMARY KEY	Unique identifier for the prediction.
ticker	VARCHAR(20)	NOT NULL	Target asset ticker symbol.

V.S.R Engineering College	FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION	Page 8
	signal	VARCHAR(10)	NOT NULL	Computed trading action (BUY/HOLD/SELL).	

6. IMPLEMENTATION - CODING

6.1 Deep Learning LSTM Network Definition

The script below, from `backend/ml/lstm\_model.py`, defines a sequential model featuring stacked LSTM recurrent layers and dropout regularization to predict price points while avoiding overfitting.

FILE: BACKEND/ML/LSTM_MODEL.PY (EXTRACT)

```
import os
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Dense
from tensorflow.keras.optimizers import Adam

def build_lstm_model():
    model = Sequential()

    # Layer 1: First LSTM layer with 128 units and 60-day historical lookback window
    model.add(LSTM(units=128, return_sequences=True, input_shape=(60, 1)))
    model.add(Dropout(0.2)) # Dropout to prevent overfitting

    # Layer 2: Second LSTM layer returning a single compressed sequence representation
    model.add(LSTM(units=64, return_sequences=False))
    model.add(Dropout(0.2))

    # Layer 3: Fully connected Dense classification layers
    model.add(Dense(units=32))
    model.add(Dense(units=1)) # Single continuous output value (predicted price)

    # Compile model with Mean Squared Error (MSE) loss function and Adam optimizer
    optimizer = Adam(learning_rate=0.001)
    model.compile(optimizer=optimizer, loss='mean_squared_error')

    return model

def save_model(model, ticker: str):
    path = os.path.join("ml", "saved_models", f"{ticker}_lstm.h5")
    model.save(path)
    print(f"Model successfully saved to {path}")
```

The LSTM model processes data in batches, passing historical parameters through memory cell loops to extract temporal associations. The output layer predicts the normalized price for the next trading day.

6. IMPLEMENTATION - CODING

6.2 Technical Indicators Computation Engine

The technical indicator engine calculates moving averages, momentum oscillators, and volatility bands on pricing streams. It uses Pandas and Pandas-TA to process datasets.

FILE: BACKEND/ML/INDICATORS.PY (EXTRACT)

```
import pandas as pd
import pandas_ta as ta

def get_all_indicators(df: pd.DataFrame) -> pd.DataFrame:
    # Ensure correct columns exist
    if 'Close' not in df.columns or 'Volume' not in df.columns:
        raise ValueError("Dataframe must contain Close and Volume columns.")

    # 1. Simple & Exponential Moving Averages
    df['SMA_20'] = ta.sma(df['Close'], length=20)
    df['EMA_50'] = ta.ema(df['Close'], length=50)

    # 2. Relative Strength Index (RSI - 14 Days)
    df['RSI_14'] = ta.rsi(df['Close'], length=14)

    # 3. Moving Average Convergence Divergence (MACD)
    macd_df = ta.macd(df['Close'], fast=12, slow=26, signal=9)
    if macd_df is not None:
        df = pd.concat([df, macd_df], axis=1)

    # 4. Bollinger Bands (20 periods, 2 standard deviations)
    bbands_df = ta.bbands(df['Close'], length=20, std=2.0)
    if bbands_df is not None:
        df = pd.concat([df, bbands_df], axis=1)

    # 5. Average True Range (ATR - Volatility indicator)
    df['ATR_14'] = ta.atr(df['High'], df['Low'], df['Close'], length=14)

    return df.fillna(0) # Standardize null values from lag calculations
```

These features serve two purposes: they populate indicators in the Angular UI Lightweight Charts layout and supply input data for classification models.

6. IMPLEMENTATION - CODING

6.3 Predictions API Router Endpoint

The FastAPI routing service handles predictions. The `/predict/{ticker}` endpoint retrieves cached predictions or executes the ML pipeline to return predictions and news sentiment metrics.

FILE: BACKEND/ROUTERS/PREDICT.PY (EXTRACT)

```
from fastapi import APIRouter, HTTPException, Depends
from sqlalchemy.orm import Session
from database import get_db
from models.db_models import Prediction
from ml.predict_utils import get_predictions
import json

router = APIRouter()

@router.get("/{ticker}")
def get_stock_predictions(ticker: str, db: Session = Depends(get_db)):
    ticker = ticker.upper().strip()

    # 1. Look for existing cached prediction in the database
    cached = db.query(Prediction).filter(Prediction.ticker == ticker).first()
    if cached:
        return {
            "ticker": ticker,
            "signal": cached.signal,
            "forecast": json.loads(cached.forecast_json),
            "confidence": cached.confidence,
            "cached": True
        }

    # 2. If cache missed, run the pipeline
    try:
        results = get_predictions(ticker)

        # 3. Store results in SQLite DB
        new_pred = Prediction(
            ticker=ticker,
            signal=results["signal"],
            forecast_json=json.dumps(results["forecast"]),
            confidence=results["confidence"]
        )
        db.add(new_pred)
        db.commit()

        return results
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Pipeline error: {str(e)}")
```

This REST design isolates model execution logic, managing load by retrieving cached items before running models.

7. OUTPUT SCREENSHOTS

7.1 Real-Time Analytics Dashboard & Charts

The image below displays the real-time stock visualization interface. The top section displays real-time candlestick feeds, volume charts, and technical indicator lines, and the sidebar displays the user's watchlist.



Dashboard Interface Screenshot

Double-click to paste or drop your `dashboard.png` file in the `report_images/` folder.

Displays: Candlestick chart panel, volume tracking, moving average overlays, and watchlist sidebar.

The chart uses canvas rendering to allow fluid panning, zooming, and tracking of historical stock trends.

7. OUTPUT SCREENSHOTS

7.2 Machine Learning Prediction & Sentiment Panel

The prediction interface displays the LSTM model forecasts and news sentiment analysis. The predictions chart displays historical trends alongside forecasted future steps.



ML Prediction Interface Screenshot

Drop your **predictions.png** file in the `report_images/` folder.



News Sentiment Interface Screenshot

Drop your **sentiment.png** file in the `report_images/` folder.

The top panel displays the model predictions, and the bottom panel displays aggregated news sentiment, highlighting positive, negative, and neutral classifications.

8. CONCLUSION & REFERENCES

8.1 Conclusion

The **Stock Market Prediction & Analysis Platform** combines quantitative charting tools with machine learning and deep learning models. By implementing stacked LSTM models, calculating technical indicators dynamically, and compiling news sentiment scores, the platform provides a unified tool for financial asset analysis.

Caching historical price records and predictions locally helps manage external API limits and minimizes data loading delays.

8.2 Limitations and Future Work

Current limitations and planned enhancements include:

- **Sentiment Channels:** Expanding sentiment extraction from news headlines to social media data sources.
- **Feature Engineering:** Incorporating macroeconomic indicators (interest rates, inflation) into the deep learning models.
- **Real-time Streaming:** Transitioning REST endpoints to WebSockets for real-time streaming data delivery.

8.3 References

- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- FastAPI Documentation: Asynchronous Python Web Framework - <https://fastapi.tiangolo.com/>
- Angular Developer Documentation: Component-based Application Architecture - <https://angular.dev/>
- Yahoo Finance API Library Reference - <https://github.com/ranaroussi/yfinance>

- Hutto, C. J. & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. *Proceedings of the International AAAI Conference on Web and Social Media*.